# Trabalho 3 (T3): Divisão e conquista com MPI

Ordenação de um vetor (Bubble Sort nas folhas) — Unidade 04

**Integrantes:** Bernardo Fioreze *(preencher demais integrantes)* | **Repositório:** https://github.com/BernardoFioreze/t3-mpi-divisao-conquista

**Ambiente:** cluster Atlântica (LAD), **2 nós exclusivos**; cada nó = 2× Intel Xeon E5520 @ 2,27 GHz = **8 núcleos físicos / 16 threads (HT) por nó** (`lscpu`), logo **16 núcleos físicos / 32 threads** nos 2 nós. $N = 200000$, 10 repetições por configuração (mediana / mín / máx), todas com `verificacao=OK`.

---

## Análise dos resultados

**1. Modelagem.** Todos os processos rodam o mesmo executável (SPMD) e descobrem seu papel pelo `rank`. Na *versão original* a árvore é indexada como um heap: a raiz é o `rank` 0, um processo $r$ tem filhos $2r{+}1$ e $2r{+}2$ e pai $(r{-}1)/2$ (`original.c`:9–10). O processo não conhece o tamanho do pedaço de antemão: ele o descobre com `MPI_Probe`/`MPI_Get_count` antes de alocar (`original.c`:99–102). Se o pedaço é $\leq \delta$ ele conquista com Bubble Sort (linhas 123–124); senão envia **as duas metades** aos filhos (linhas 129–130) **antes** de qualquer `MPI_Recv` (linhas 132–133), para os dois filhos trabalharem ao mesmo tempo, e intercala na volta (linha 134). Na *versão balanceada* o processo não entrega as duas metades: manda a metade direita ao ajudante $r + 2^{\text{nível}}$ (`MPI_Isend`, `balanceado.c`:90) e **fica com a esquerda**, descendo um nível (linha 95); recebe a direita ordenada de volta e intercala (linhas 98–100). Um não-raiz descobre onde nasceu pelo bit menos significativo do `rank`: nível inicial = $\texttt{ctz}(r) - 1$ (linha 170) e pai = $r \,\&\, (r{-}1)$ (linha 139). O `MPI_Isend` seguido de `MPI_Request_free` (linhas 90–92) evita bloquear e descarta a *request* porque o `MPI_Recv` do mesmo ajudante (linha 98) só retorna depois que a mensagem foi consumida. Com $P$ potência de 2 todas as folhas têm $N/P$; com $P = 2^k - 1$ (caso da original) elas diferem em um elemento.

**2. Speed-up, eficiência e a separação do ganho de algoritmo.** A Tabela 1 mostra o ganho de algoritmo $T_1/T_1^{\delta}$ indo de 2,0× ($\delta$=$10^5$) a 535× ($\delta$=391): quase todo o "speed-up" acima de $P$ vem daí, não do paralelismo. Por exemplo, na original com $P$=31 e $\delta$=12500 temos $S = 125\times$ mas $S^{\delta} = 7{,}4\times$; e 7,4 × 16,9 (o ganho de algoritmo em $\delta$=12500) $\approx 125$. A $S^{\delta}$ é o paralelismo honesto (contra a *mesma* árvore em um processo) e a eficiência $E = S^{\delta}/P$ cai de 0,67 ($P$=3) a 0,24 ($P$=31) por causa da comunicação e da árvore (só as folhas calculam — ponto 3). Para a versão balanceada com **31 processos** (não potência de 2), as folhas têm tamanhos diferentes e um $\delta$ único não reproduz a árvore; por isso ela fica fora da Tabela 2 (que traz só as potências de 2, com $S^{\delta}$ e $E$), mas seu $S$ é válido e medido: $T_P = 0{,}205$ s, $S = 197\times$, contra 125× da original no *mesmo* $P$=31 (ponto 5).

**3. Balanceamento da carga nos níveis (original).** Cada nó interno divide e então fica **parado** em `MPI_Recv` (`original.c`:132–133) até os filhos voltarem; apenas o nível das folhas executa Bubble Sort. Num dado instante os níveis superiores estão ociosos e, na volta, fazem só a intercalação barata ($O(n)$). É esse desbalanceamento que a versão balanceada ataca ao fazer o pai também ordenar uma folha.

**4. Ganho com HT.** Comparando $P$=15 (um processo por núcleo físico; há 16 físicos nos 2 nós) com $P$=31 (passa a usar as threads de Hyper-Threading): o tempo cai de 0,709 para 0,322 s (2,2×), mas o $\delta$ também muda (25000→12500). Olhando a $S^{\delta}$, que isola o paralelismo, o ganho é só 6,93→7,41 (+7%). Ou seja, o HT quase não aumenta o paralelismo real: as duas threads de um núcleo dividem as mesmas unidades de execução e o Bubble Sort é limitado por CPU; o grosso da melhora de $P$=15 para 31 vem das folhas menores (algoritmo), não de mais processamento simultâneo.

**5. Ganho com o balanceamento.** Duas formas, como pede o enunciado. *Sem alterar o código* (mais processos que núcleos — série de oversubscription, Tabela 3): quem balanceia é o escalonador do SO (ponto 6). *Alterando o código* (versão balanceada): no mesmo $P$=31 o tempo cai de 0,322 para 0,205 s (1,57× mais rápido) porque os processos internos deixam de ficar ociosos e também ordenam uma folha. Em potências de 2 a eficiência da balanceada se mantém alta ($E \approx 0{,}49$ até $P$=8) enquanto a da original já está caindo.

**6. Desempenho de 63 a 1023 processos: espera ativa × núcleo cedido.** Com mais processos que as 32 threads de hardware, a **espera ativa** (`OMPI_MCA_mpi_yield_when_idle=0`) faz um processo bloqueado em `MPI_Recv` ocupar o núcleo em laço; com `OMPI_MCA_mpi_yield_when_idle=1` ele **cede o núcleo**. As duas séries foram medidas com a variável **definida explicitamente** (0 e 1), como pede a errata do enunciado, pois deixá-la indefinida pode já ceder o núcleo e fazer as duas medirem a mesma coisa (foi o que observamos: com a variável indefinida a espera ativa era bem mais rápida, sinal de que o núcleo já estava sendo cedido). Em oversubscription moderado o *yield* ganha claramente: em $P$=127, 0,519 s (espera ativa) contra 0,184 s (cede o núcleo), 2,8×; em $P$=63, 0,333 contra 0,214. A diferença entre as duas séries é exatamente quanto o escalonador do SO consegue balancear quando os processos cooperam cedendo o núcleo que não estão usando. A partir de $P$=255 os dois pioram (muito além de 32 threads: troca de contexto e pressão de memória dominam) e a eficiência $E = S^{\delta}/32$ fica ínfima, mas o *yield* degrada menos. O caso $P$=1023 rodou completo nos dois modos após elevar `ulimit -n` a 512000 e usar `opal_set_max_sys_limits` (senão estourava o limite de *pipes* do SO ao encaminhar a saída de cada rank).

**7. Uso de ferramentas de IA.** Declaramos o uso do Claude (Anthropic) para: entender o enunciado; derivar a implementação a partir do pseudocódigo (indexação em heap na original; esquema de "pente"/Radenski na balanceada); escrever os scripts `sbatch` e o `analise.py`; e depurar erros do cluster (falha do PMIx e o limite de *pipes* em $P$=1023). Revisamos o código linha a linha, sabemos explicar cada chamada MPI, e as decisões de modelagem e de medição são do grupo.

Tabela 1: Referências sequenciais ($N$=200000, 1 processo). A coluna *níveis* é o número de níveis da árvore (nível 1 = vetor inteiro, sem divisão, $\delta = N$ — é o próprio $T_1$); cada nível a mais divide as folhas pela metade. Todas rodam em um só processo. A mesma árvore de nível $n$ corresponde a $P = 2^n - 1$ processos na versão original e $P = 2^{n-1}$ na balanceada (daí os $\delta$ compartilhados na Tabela 2). O ganho de algoritmo $T_1/T_1^\delta$ explica por que $S$ passa de $P$.

| nível | $\delta$ | mediana (s) | mín (s) | máx (s) | $T_1/T_1^\delta$ |
|---|---|---|---|---|---|
| 1 ($T_1$) | 200000 | 40,354 | 40,339 | 40,383 | 1,00 |
| 2 | 100000 | 20,051 | 20,033 | 20,073 | 2,01 |
| 3 | 50000 | 9,961 | 9,913 | 9,975 | 4,05 |
| 4 | 25000 | 4,912 | 4,904 | 4,921 | 8,22 |
| 5 | 12500 | 2,384 | 2,379 | 2,387 | 16,93 |
| 6 | 6250 | 1,125 | 1,113 | 1,134 | 35,87 |
| 7 | 3125 | 0,552 | 0,547 | 0,562 | 73,14 |
| 8 | 1563 | 0,290 | 0,287 | 0,292 | 139,35 |
| 9 | 782 | 0,152 | 0,149 | 0,155 | 265,23 |
| 10 | 391 | 0,075 | 0,075 | 0,081 | 535,04 |

Tabela 2: Original ($P = 2^n - 1$) e balanceada ($P = 2^{n-1}$) lado a lado, por nível $n$ da árvore. No mesmo nível as duas ordenam folhas do **mesmo tamanho** ($\delta = N/2^{n-1}$, coluna única); o que muda é quantos processos cada uma usa para chegar lá. $S = T_1/T_P$; $S^\delta = T_1^\delta/T_P$; $E = S^\delta/P$; tempos em segundos. O nível 1 (original com $P$=1, vetor inteiro) coincide com $T_1$ da Tabela 1 ($T_P$=40,39, $S$=1,00); a balanceada começa no nível 2 e a original não alcança o nível 6.

| | | **original** | | | | | **balanceada** | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| nível | $\delta$ | $P$ | $T_P$ | $S$ | $S^\delta$ | $E$ | $P$ | $T_P$ | $S$ | $S^\delta$ | $E$ |
| 2 | 100000 | 3 | 10,053 | 4,01 | 1,99 | 0,665 | 2 | 20,081 | 2,01 | 1,00 | 0,499 |
| 3 | 50000 | 7 | 2,577 | 15,66 | 3,87 | 0,552 | 4 | 4,992 | 8,08 | 1,99 | 0,499 |
| 4 | 25000 | 15 | 0,709 | 56,90 | 6,93 | 0,462 | 8 | 1,262 | 31,98 | 3,90 | 0,487 |
| 5 | 12500 | 31 | 0,322 | 125,38 | 7,41 | 0,239 | 16 | 0,361 | 111,84 | 6,61 | 0,413 |
| 6 | 6250 | — (original vai só até $P$=31) — | | | | | 32 | 0,170 | 236,72 | 6,56 | 0,205 |

Tabela 3: Oversubscription da versão original (63–1023 processos nos mesmos 2 nós = 32 threads). Duas execuções, com a variável definida explicitamente: espera ativa (`OMPI_MCA_mpi_yield_when_idle=0`) e cede o núcleo (`=1`). $E = S^\delta/32$ (eficiência relativa às 32 threads de hardware).

| | espera ativa (`yield_when_idle=0`) | | | | | cede o núcleo (`=1`) | | | |
|---|---|---|---|---|---|---|---|---|---|
| $P$ | $\delta$ | $T_P$ (s) | $S$ | $S^\delta$ | $E$ | $T_P$ (s) | $S$ | $S^\delta$ | $E$ |
| 63 | 6250 | 0,333 | 121,2 | 3,38 | 0,106 | 0,214 | 188,3 | 5,25 | 0,164 |
| 127 | 3125 | 0,519 | 77,7 | 1,06 | 0,033 | 0,184 | 219,8 | 3,01 | 0,094 |
| 255 | 1563 | 0,728 | 55,4 | 0,40 | 0,012 | 0,228 | 177,2 | 1,27 | 0,040 |
| 511 | 782 | 1,906 | 21,2 | 0,08 | 0,002 | 0,427 | 94,5 | 0,36 | 0,011 |
| 1023 | 391 | 6,534 | 6,2 | 0,01 | 0,000 | 2,119 | 19,1 | 0,04 | 0,001 |

O código completo (`src/sequencial.c`, `src/original.c`, `src/balanceado.c`), os scripts `sbatch` e as saídas brutas do Slurm estão no repositório público citado no cabeçalho; as referências de linha acima apontam para esses arquivos.